

MomAIOS

Documentacao Completa do Projeto

MomAI — Assistente Virtual Local-First com Foco em Privacidade

Gerado em: 08/05/2026 22:50

Versao: 1.3.0

Licenca: MIT

Autor: WesleyQDev

Este documento apresenta uma visao abrangente do projeto MomAIOS, cobrindo arquitetura, tecnologias, decisoes tecnicas, fluxos de dados, estrutura de diretorios, guia de desenvolvimento e muito mais. Todo o conteudo foi gerado a partir da analise do codigo-fonte e documentacao existente no repositorio.

Sumario

1. Visao Geral do Projeto

- 1.1 O que é o MomAIOS
- 1.2 Público-Alvo
- 1.3 Filosofia de Design
- 1.4 Arquitetura Geral

2. Tecnologias e Stack

- 2.1 Tabela Completa de Tecnologias
- 2.2 Por que Cada Tecnologia Foi Escolhida

3. Decisões Técnicas Importantes

- 3.1 LLM Local vs Cloud
- 3.2 Node.js vs Python para Orquestração
- 3.3 Sidecar Python para Voz
- 3.4 Sistema de Tiers (Lite, Pro, Ultra)
- 3.5 pnpm + Turborepo

4. Estrutura de Diretórios

- 4.1 Árvore Comentada

5. Principais Funcionalidades e Fluxos

- 5.1 Pipeline de Voz
- 5.2 Respostas Estruturadas
- 5.3 Sistema de Extensões
- 5.4 Modo Chamada (Call Mode)

6. Guia de Desenvolvimento

- 6.1 Setup do Ambiente
- 6.2 Comandos Essenciais
- 6.3 Convenções de Código
- 6.4 Testes
- 6.5 Build e Release

7. Dependências e Bibliotecas

- 7.1 Dependências Críticas do Desktop
- 7.2 Dependências Críticas do Core

8. Configuração e Ambiente

- 8.1 Variáveis de Ambiente
- 8.2 Arquivos de Configuração
- 8.3 CI/CD Pipeline

9. Glossário

1. Visao Geral do Projeto

1.1 O que e o MomAIOS

MomAIOS e o monorepo que abriga o MomAI, um assistente virtual de desktop que combina a inteligencia de Modelos de Linguagem de Grande Porte (LLMs) com a capacidade de executar acoes reais no computador do usuario. Diferente de assistentes como Alexa, Google Assistant ou Siri, o MomAI foi projetado desde o inicio com um principio fundamental: privacidade em primeiro lugar. Todo o processamento, desde a deteccao da palavra de ativacao ate a geracao de respostas em texto e fala, acontece localmente na maquina do usuario.

O projeto e mantido por WesleyQDev e esta licenciado sob MIT, sendo totalmente gratuito e de codigo aberto.

1.2 Publico-Alvo

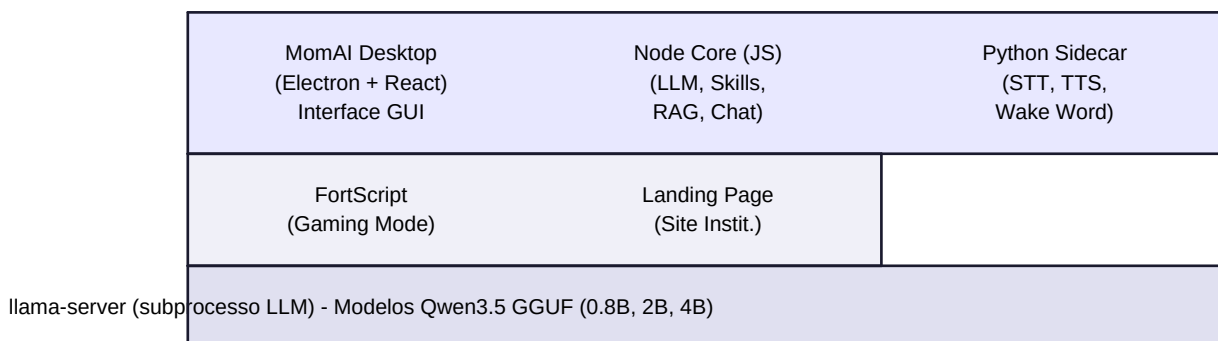
O MomAI e direcionado a usuarios preocupados com privacidade que nao querem que seus dados de voz e conversas sejam enviados para servidores cloud. Tambem atende desenvolvedores que podem estender o assistente com novas capacidades atraves do sistema de extensoes, entusiastas de IA que querem executar modelos de linguagem localmente, e usuarios de PC em geral que buscam uma assistente que vai alem do basico, com integracao a notas, lembretes, pesquisa web e acoes no sistema.

1.3 Filosofia de Design

O MomAI foi construido sobre tres pilares fundamentais. O primeiro e Local-First: tudo roda localmente e nenhum dado sai da maquina do usuario sem permissao explicita, como no caso da pesquisa web. O segundo e Extensivel: o sistema de extensoes permite que qualquer pessoa adicione novas capacidades, desde ferramentas simples para o LLM ate interfaces completas na barra lateral. O terceiro e Modular: a arquitetura com Node Core e Python Sidecar separados permite que cada componente evolua independentemente, facilitando a manutencao e os testes.

1.4 Arquitetura Geral

O MomAIOS e organizado como um monorepo gerenciado por pnpm workspaces e Turborepo. Isso significa que multiplos aplicativos e bibliotecas vivem no mesmo repositorio, compartilhando configuracoes de build, lint e teste, mas cada um com seu proprio ciclo de vida e dependencias. O diagrama a seguir ilustra a arquitetura em alto nivel:



Legenda: As setas indicam comunicacao via HTTP/WebSocket. O Electron Main Process gerencia Node Core e Python Sidecar como subprocessos.

A comunicacao entre os componentes segue uma hierarquia clara. O Electron Main Process inicia e gerencia tanto o Node Core quanto o Python Sidecar como subprocessos. O Node Core (porta 8000) e o cerebro da IA: gerencia o LLM, skills, RAG semantico e chat. O Python Sidecar (porta 8001) lida exclusivamente com audio: transcricao (STT), sintese de fala (TTS) e deteccao de palavra de ativacao. O llama-server (porta 8080) e o processo que realmente carrega o modelo GGUF e executa as inferencias. O Renderer React se comunica com o Node Core via HTTP e SSE streaming para respostas em tempo real, e com o Python Sidecar via HTTP e WebSocket para operacoes de voz.

2. Tecnologias e Stack

2.1 Tabela Completa de Tecnologias

Tecnologia	Versao	App	Proposito
Electron	39.x	Desktop	Container desktop multiplataforma
React	19.x	Desktop, Landing	Interface do usuario
TypeScript	5.9.x	Desktop, Landing	Type safety
TailwindCSS	3.4.x	Desktop, Landing	Estilizacao utility-first
Vite	7.x	Desktop, Landing	Bundler e dev server
electron-vite	5.x	Desktop	Bundling Electron
electron-builder	26.x	Desktop	Packaging (NSIS, DMG, ApplImage)
LangGraph	1.2.x	Node Core	Orientacao de agentes
LangChain Core	1.1.x	Node Core	Framework de LLM
LanceDB	0.27.x	Node Core	Banco vetorial embedded
Python	3.12+	Core	Runtime sidecar de voz
FastAPI	0.128+	Core	Framework web assincrono
faster-whisper	1.2.x	Core	STT (transcricao) via CTranslate2
kokoro-onnx	0.5+	Core	TTS (síntese) via ONNX
pnpm	10.28	Root	Gerenciador de pacotes
Turborepo	2.7	Root	Build system do monorepo

2.2 Por que Cada Tecnologia Foi Escolhida

Electron foi escolhido como container desktop porque permite que uma base de código React/TypeScript rode em Windows, Linux e macOS sem modificações. Sua maturidade de mais de uma década garante estabilidade e um ecossistema vasto de ferramentas como electron-builder e electron-updater.

pnpm + Turborepo formam a espinha dorsal do monorepo. pnpm foi escolhido sobre npm e yarn por seu uso eficiente de disco (hard links) e workspaces nativos. Turborepo adiciona cache inteligente de build, paralelismo e dependências entre tarefas sem a complexidade do Nx.

LanceDB e o banco vetorial escolhido para memória semântica. Diferente de soluções como Pinecone (cloud) ou pgvector (requer PostgreSQL), o LanceDB roda embedded dentro do processo Node.js, não requer configuração de servidor, e é otimizado especificamente para busca vetorial.

LangGraph foi escolhido para orquestração de agentes porque oferece controle fino sobre fluxos de conversação. Diferente de cadeias lineares, o LangGraph permite grafos cíclicos, estados

compartilhados entre passos e branchings condicionais.

Kokoro-82m como engine TTS roda via ONNX Runtime em CPU sem necessidade de GPU. Sua qualidade de voz é comparável a soluções cloud como Google Cloud TTS.

faster-whisper (CTranslate2) foi escolhido sobre whisper.cpp para STT por oferecer melhor performance em CPU.

uv como gerenciador Python foi escolhido por sua velocidade (Rust), compatibilidade com pip e instalação simplificada.

3. Decisoese Tecnicas Importantes

3.1 LLM Local vs Cloud

O problema central era como fornecer inteligencia de LLM sem comprometer a privacidade do usuario. As alternativas consideradas foram: API OpenAI/Anthropic (cloud, facil, mas dados vao para servidores externos), llama.cpp via subprocesso (local, complexo, mas privado), e ONNX Runtime com modelos convertidos (local, mas perda de compatibilidade).

A escolha foi llama.cpp via subprocesso (llama-server.exe). O llama-server e um executavel maduro que implementa a API compativel com OpenAI, permitindo que o Node Core use a mesma interface que usaria para chamar a API da OpenAI, mas apontando para localhost:8080. Isso permite que o codigo trate LLM local e cloud de forma intercambiavel. O trade-off principal e que requer download de modelos de 1 a 4 GB cada e consome RAM/VRAM significativa.

3.2 Node.js vs Python para Orquestracao

Como o Node Core roda como subprocesso do Electron, usar Node.js elimina a necessidade de bridges IPC complexas entre runtimes diferentes. O LangChain tem suporte completo a Node.js e bibliotecas como LanceDB tambem tem bindings nativos. A comunicacao entre o renderer e o Node Core e direta via HTTP na mesma porta. O trade-off e que o ecossistema de IA em Node.js e menos maduro que em Python, o que motivou a criacao do Python Sidecar para operacoes de voz.

3.3 Sidecar Python para Voz

STT e TTS exigem bibliotecas Python especializadas sem equivalentes Node.js maduros. Em vez de executar Python inline no Node.js (fragil e dificil de debugar), a escolha foi um sidecar separado com FastAPI. Isso oferece isolamento de processo, permite reinicializacao independente, e comunicacao padronizada via HTTP/WebSocket. O Python Sidecar e enxuto (aproximadamente 900 linhas de codigo) e tem responsabilidade unica: operacoes de voz.

3.4 Sistema de Tiers (Lite, Pro, Ultra)

Os tiers permitem que o usuario escolha entre performance (Lite: Qwen3.5-0.8B, 192 tokens, sem TTS) e qualidade (Ultra: Qwen3.5-4B, 512 tokens, TTS + wake word + memoria vetorial). Isso e essencial porque um laptop com 8GB de RAM nao consegue rodar o mesmo modelo que um desktop com 32GB. A configuracao em ai_tiers.json torna facil adicionar novos tiers ou modelos no futuro. O tier padrao e Pro (Qwen3.5-2B, 320 tokens, TTS ativado).

Decisao	Escolha	Alternativa	Motivo
LLM Runtime	llama-server (subprocesso)	API externa (OpenAI)	Privacidade total, baixa latencia
Embeddings	LanceDB (local)	PostgreSQL pgvector	Zero-config, embedded
TTS	Kokoro (ONNX) via Python	API Cloud (Google, AWS)	Offline, qualidade alta

Decisao	Escolha	Alternativa	Motivo
Orientacao de Agentes	LangGraph	CrewAI, AutoGen	Controle fino de fluxo
Skill Runtime	Node.js	Python	Mesmo runtime do Electron
Build System	pnpm + Turborepo	Nx, Lerna	Simplicidade, workspaces nativos
Ger. Python	uv	Poetry, pipenv	Velocidade (Rust)

4. Estrutura de Diretórios

4.1 Árvore Comentada

A estrutura de diretórios reflete a organização em monorepo com pnpm workspaces. Cada aplicação tem seu próprio diretório em apps/ com suas dependências e configuração independentes. O diretório scripts/ contém scripts de build e utilitários. docs/ abriga a documentação técnica.

```
momai/ +- apps/ | +- core/ Python Sidecar (STT, TTS, Wake Word) | +- fortscript/ Gaming mode process manager | +- landing-page/ Site institucional (Vite + React) | +- momai/ Desktop App (Electron + React) | | +- src/main/ Electron Main Process | | +- src/preload/ Bridge segura | | +- src/renderer/ React SPA | | +- scripts/ Node Core + Skills + Build scripts | | +- electron-builder.yml | +- momai-promo-video/ Video promocional (Remotion) +- scripts/ Scripts raiz +- docs/ Documentação técnica +- .github/workflows/ CI/CD (CI, Release, Deploy) +- package.json Root package.json +- pnpm-workspace.yaml Config workspaces +- turbo.json Config Turborepo
```

No coração do projeto está o diretório apps/momai/, que contém o aplicativo desktop. Sua estrutura interna é dividida entre os três processos do Electron (main, preload, renderer) e os scripts auxiliares. O Node Core, originalmente um único arquivo de 4.432 linhas, foi refatorado para módulos em scripts/node-core/ com serviços bem definidos como chat-service, llama-manager e skill-orchestrator.

5. Principais Funcionalidades e Fluxos

5.1 Pipeline de Voz

O pipeline de voz é uma das funcionalidades mais complexas do MomAI, envolvendo todos os componentes do sistema. O fluxo começa com o microfone capturando áudio continuamente. O OpenWakeWord Detector, rodando no Python Sidecar 100% offline, detecta a palavra-chave "Sistema" (ou "Luna", "Computador") em menos de 100ms. Após a detecção, o sistema inicia gravação do microfone e o faster-whisper transcreve o áudio para texto. O texto transcrito é enviado para o Node Core, que processa o comando de voz através do LLM e gera uma resposta. Se o TTS estiver ativado e o tier for Pro ou Ultra, a resposta é enviada para o Kokoro TTS no Python Sidecar, que sintetiza a fala e toca nos alto-falantes.

```
[Microfone] -> [Wake Word Detector (OpenWakeWord)] -> [faster-whisper STT] -> [Node Core (LLM)] -> [Renderer (texto)] + [Kokoro TTS (audio)]
```

5.2 Respostas Estruturadas (Structured Skill Responses)

Skills podem retornar componentes de interface ricos em vez de texto plano. Quando uma skill retorna um structuredResponse, o Node Core serializa como JSON e envia via SSE. O frontend recebe o evento, consulta o SkillResponseRegistry para encontrar o renderizador registrado para aquele tipo, e renderiza o componente visual apropriado. Por exemplo, a skill de clima retorna um objeto com type "weather" e o WeatherCard exibe um card com previsão de 7 dias, emojis e temperaturas. O registro é flexível: qualquer componente pode ser registrado com registerRenderer(type, Component).

5.3 Sistema de Extensões

O MomAI suporta três tipos de extensões. Skills Built-in são nativas em scripts/skills/core/ e rodam no mesmo processo do Node Core. Skills Packaged estão em scripts/skills/packaged/ e são executadas em host isolado. Extensions são instaladas pelo usuário via loja de extensões (ExtensionsView.tsx) e também executadas em ambiente isolado com sistema de permissões declarativas. O manifesto da extensão (manifest.json) define permissões de rede, sistema de arquivos e subprocesso, que são verificados antes da execução.

5.4 Modo Chamada (Call Mode)

O call mode é um modo mãos-livres onde o usuário pode conversar com o MomAI sem precisar digitar ou ativar por wake word a cada interação. O wake word detector continua rodando mas sem filtro de keyword, processando qualquer fala detectada. Uma janela overlay transparente mostra o texto em tempo real. O call mode é desativado quando o usuário diz "tchau" ou desativa manualmente.

6. Guia de Desenvolvimento

6.1 Setup do Ambiente

Os pre-requisitos sao Node.js 20+, pnpm 9+ (instalado globalmente via npm), Python 3.12+ e Git. Apos clonar o repositorio, execute pnpm install para instalar todas as dependencias do monorepo. Para iniciar o desenvolvimento completo (desktop + backend), use pnpm dev:all. Na primeira execucao, o script ensure-dev-binaries.js baixara automaticamente os binarios necessarios como llama-server, Python bundled e uv.

6.2 Comandos Essenciais

Comando	Descricao
pnpm dev:all	Desktop + Core simultaneos
pnpm dev:core	Apenas o Python backend
pnpm --filter momai dev	Apenas o desktop app
pnpm build	Build completo
pnpm build:win	Build Windows .exe (NSIS)
pnpm build:linux	Build Linux AppImage
pnpm lint	Lint de todas as apps
pnpm typecheck	TypeScript check
pnpm test	Testes
pnpm format	Prettier

6.3 Convencoes de Codigo

Em TypeScript/React, componentes usam PascalCase, hooks usam prefixo use com camelCase, utilitarios usam camelCase, constantes usam UPPER_SNAKE_CASE e arquivos usam kebab-case. Testes ficam ao lado do arquivo testado com extensao .test.ts. Em Python, segue-se PEP 8 com type hints obrigatorios, async/await para I/O, snake_case para funcoes e PascalCase para classes. No FastAPI, usa-se Depends() para injecao de dependencia e schemas Pydantic para validacao.

6.4 Testes

O desktop usa Vitest para testes unitarios, com suporte a testes do renderer (React) e do main process (Node.js). O core usa pytest com pytest-asyncio para testes assincronos. Para rodar todos os testes do monorepo, use pnpm test na raiz.

6.5 Build e Release

O processo de release e semi-automatizado via GitHub Actions. Ao criar uma tag v1.2.3 no git, o CI detecta a tag e dispara o workflow de release. O build para Windows e Linux roda em paralelo em maquinas diferentes. Wheels Python sao pre-compiladas durante o build usando uv pip compile. O release final e publicado no repositorio publico WesleyQDev/MomAI-App com todos os artefatos de instalacao.

7. Dependencias e Bibliotecas

7.1 Dependencias Criticas do Desktop

O aplicativo desktop (apps/momai/) tem cerca de 30 dependencias principais. As mais criticas sao Electron (container desktop), React (interface), @lancedb/lancedb (banco vetorial), @langchain/core e @langchain/langgraph (framework de IA), axios (HTTP), electron-updater (auto-update) e ws (WebSocket).

Biblioteca	Versao	Proposito	Nivel
electron	39.x	Container desktop	Essencial
react	19.x	Interface do usuario	Essencial
@lancedb/lancedb	0.27.x	Banco vetorial local	Essencial
@langchain/core	1.1.x	Framework LLM	Essencial
@langchain/langgraph	1.2.x	Orientacao agentes	Essencial
electron-updater	6.8.x	Auto-update	Essencial
ws	8.x	WebSocket	Essencial
axios	1.x	Cliente HTTP	Essencial
edge-tts-universal	1.4.x	TTS fallback	Importante
zod	4.x	Validacao schemas	Utilitario

7.2 Dependencias Criticas do Core

O Python Sidecar (apps/core/) tem 11 dependencias principais. As mais criticas sao fastapi (framework web), faster-whisper e ctranslate2 (STT), kokoro-onnx e onnxruntime (TTS), sounddevice (captura de audio) e sqlalchemy (ORM).

Biblioteca	Versao	Proposito	Nivel
fastapi	0.128+	Framework web	Essencial
faster-whisper	1.2.x	STT (transcricao)	Essencial
kokoro-onnx	0.5+	TTS (síntese de fala)	Essencial
onnxruntime	1.20+	Runtime ONNX	Essencial
ctranslate2	4.4.x	Runtime Whisper	Essencial
sounddevice	0.5+	Captura de audio	Essencial
sqlalchemy	2.0+	ORM	Essencial
huggingface-hub	1.3+	Download modelos	Essencial

Biblioteca	Versao	Proposito	Nivel
httpx	0.28+	Cliente HTTP async	Importante
psutil	7.2+	Monitoramento	Importante

8. Configuracao e Ambiente

8.1 Variaveis de Ambiente

O MomAI utiliza variaveis de ambiente para configuracao em tempo de execucao. O desktop usa um arquivo .env em apps/momai/, enquanto o core usa apps/core/.env. As principais variaveis controlam as portas dos servicos, diretorios de dados e modos de debug.

Variavel	Default	Descricao
MOMAI_NODE_CORE_HOST	127.0.0.1	Host do Node Core
MOMAI_NODE_CORE_PORT	8000	Porta do Node Core
MOMAI_PYTHON_SIDEAR_PORT	8001	Porta do Python Sidecar
MOMAI_LLAMA_PORT	8080	Porta do llama-server
MOMAI_EMBEDDING_PORT	8081	Porta do servidor embeddings
MOMAI_MODELS_DIR	apps/core/models/	Diretorio de modelos GGUF
MOMAI_DEBUG	false	Modo debug
LOG_LEVEL	info	Nivel de logging

8.2 Arquivos de Configuracao

O projeto usa varios arquivos de configuracao. electron-builder.yml define o packaging do desktop, electron.vite.config.ts configura o Vite para Electron, ai_tiers.json define os modelos e parametros de cada tier de IA, e pyproject.toml gerencia as dependencias Python. O turbo.json e pnpm-workspace.yaml configuram o build system do monorepo.

8.3 CI/CD Pipeline

O projeto tem tres workflows do GitHub Actions. O CI (ci.yml) executa lint e typecheck em pushes e PRs para main/develop. O Release (release.yml) e acionado por tags v*.* e constroi o aplicativo para Windows e Linux em paralelo, publicando os artefatos no repositorio publico WesleyQDev/MomAI-App. O Deploy Landing Page (deploy-landing.yml) constroi e publica o site institucional no GitHub Pages.

9. Glossario

Este glossario reúne os principais termos técnicos utilizados no projeto MomAIOS.

Termo	Definição
LLM	Large Language Model - Modelo de Linguagem de Grande Porte
STT	Speech-to-Text - Tecnologia que converte fala em texto
TTS	Text-to-Speech - Tecnologia que converte texto em fala
RAG	Retrieval-Augmented Generation - Busca informação antes de gerar resposta
SSE	Server-Sent Events - Stream unidirecional do servidor para o cliente
GGUF	Formato de arquivo de modelo otimizado para llama.cpp
ONNX	Open Neural Network Exchange - Formato aberto de modelos de ML
Tier	Nível de serviço (Lite, Pro, Ultra) com diferentes capacidades
Sidecar	Processo auxiliar que roda junto com o processo principal
Wake Word	Palavra de ativação que "acorda" o assistente (ex: Sistema)
Embedding	Vetor numérico que representa o significado de um texto
LangGraph	Framework da LangChain para orquestração de agentes em grafo
Monorepo	Único repositório contendo múltiplos projetos independentes
IPC	Inter-Process Communication - Comunicação entre processos
CTranslate2	Runtime otimizado para inferência de modelos Transformer
Kokoro	Modelo de TTS de alta qualidade que roda via ONNX
Whisper	Modelo de STT da OpenAI para transcrição de áudio

--- Fim da Documentação ---